Road Damage Detection and Classification

Perawis Buranasing
Faculty of ICT, Mahidol University,
Nakhon Pathom, Thailand 73170
perawis.bur@student.mahidol.ac.th

Atichat Kangsamut
Faculty of ICT, Mahidol University,
Nakhon Pathom, Thailand 73170
atichat.kan@student.mahidol.ac.th

Abstract—This paper presents an iterative development of a road damage detection and classification system using deep learning on the RDD2022 dataset (Japan subset). The dataset contains four damage categories: longitudinal crack (D00), transverse crack (D10), alligator crack (D20), and pothole (D40). We train three architecturally different models in sequence, where each experiment is motivated by specific weaknesses identified from the previous model. Starting from an SSD baseline (mAP@0.5 = 0.210), we upgrade to Faster R-CNN with ResNet-50 FPN (mAP@0.5 = 0.561, +167%), then to YOLOv8s which matches Faster R-CNN accuracy while being approximately 16× faster at only 11 ms per image. A controlled single-variable ablation confirms that increasing input resolution from 640 to 832 pixels specifically improves thin-crack detection (D10 AP +0.023), while having near-zero effect on wider damage classes. Our final model achieves mAP@0.5 = 0.562 and is deployed as a Gradio web application supporting image upload, video processing, and live webcam inference.

Index Terms—road damage detection, object detection, YOLOv8, Faster R-CNN, SSD, RDD2022, deep learning

I. INTRODUCTION

Road infrastructure is essential for transportation safety and economic productivity. When road surfaces deteriorate from traffic loads, weather, or aging, they create hazards for drivers and increase vehicle maintenance costs. Traditional inspection methods rely on human surveyors who physically examine roads, an approach that is expensive, time-consuming, and produces inconsistent results depending on the inspector. Automated detection of road damage from camera images would allow road authorities to monitor large road networks efficiently and prioritize repairs based on objective data, a direction that competitive benchmarking in prior work [10] has shown to be technically feasible with modern CNN-based detectors.

Recent advances in deep learning have made this possible through object detection, which simultaneously localizes and classifies objects in an image. Unlike simple image classification, object detection identifies where each damage instance is located (bounding box) and what type it is (class label with confidence score). This localization information is crucial for practical use: a system that marks the exact position of a pothole or crack on a road image gives maintenance crews actionable information.

In this work, we develop a road damage detector on the RDD2022 benchmark dataset [4] using the Japan subset, which contains four damage types: D00 (longitudinal crack), D10 (transverse crack), D20 (alligator crack), and D40 (pothole). We train three architecturally different detection models, namely SSD [1], Faster R-CNN [2], and YOLOv8s [3], in an iterative manner where each experiment is motivated by specific, measurable weaknesses identified in the previous model. A fourth experiment, which serves as an ablation study, tests whether input resolution affects thin-crack detection. Every decision is supported by quantitative evidence, including learning curves, per-class AP, localization ratios, and inference speed.

This work makes three concrete contributions: (1) a three-architecture comparison (SSD → Faster R-CNN → YOLOv8s) where each experiment is driven by specific, measured weaknesses from the previous model rather than arbitrary choices; (2) an input-resolution ablation that quantifies how much resolution matters for detecting thin transverse cracks (D10) versus wider damage types; and (3) a deployed Gradio web application supporting real-time inference via image upload, video processing, and live webcam.

II. METHODOLOGY

A. Dataset

The Road Damage Dataset 2022 (RDD2022) [4] contains 47,420 road images collected from six countries, namely Japan, India, Czech Republic, Norway, the United States, and China (with the latter divided into two subsets based on collection vehicle type), using smartphone cameras mounted on vehicles. Images are annotated in Pascal VOC XML format [8] with four official damage classes: D00 (longitudinal crack), D10 (transverse crack), D20 (alligator crack), and D40 (pothole). For this project, we use the Japan subset because it is the largest single-country collection (10,506 images), uses consistent vehicle-mounted cameras that match real deployment conditions, and provides the best label quality.

During initial dataset inspection, we discovered several undocumented label codes in the Japan subset that are not described in the official RDD2022 paper: D43 (736 objects), D44 (3,995 objects), and D50 (3,553 objects). These appear to be Japan-specific road authority codes for crosswalk blur, white line blur, and manhole covers, respectively, none of which are valid road damage classes for this task. All objects with unknown labels were filtered out before training to avoid noisy supervision.

Since the official test split contains zero XML annotation files across all countries, we created our own validation set using a stratified 80/20 split (seed=42) on the training data. After filtering, the final dataset contains 8,407 training images and 2,099 validation images with 13,071 and 3,398 annotated objects respectively. Background-only images (no annotated objects) are retained to help the model suppress false positives.

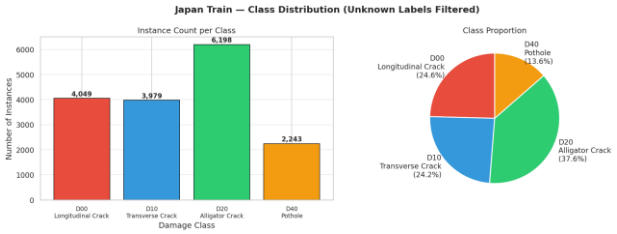


Fig. 1. Class distribution of valid objects in the Japan subset after filtering. D40 (pothole) is the minority class at 13.6% of all annotated objects.

The class distribution after filtering is: D00 24.6%, D10 24.2%, D20 37.6%, D40 13.6%. D40 (pothole) is notably underrepresented, which we monitor closely across all experiments to verify the model generalizes to minority classes.

B. Model Development

We follow an iterative development approach throughout: start with a simple baseline, rigorously identify its weaknesses from the training evidence, then design the next experiment to target exactly those weaknesses. In practice, this meant the results from each experiment genuinely shaped what we tried next, and the SSD results, for example, influenced our choice of architecture for Experiment 2 in a way we had not fully anticipated before seeing the data. For each experiment, we report learning curves, detection metrics, and a concrete analysis of the problems that motivate the following step.

Experiment 1 — SSD Baseline. We chose SSDLite320 with MobileNetV3-Large backbone [5] as the starting point. The motivation is to establish a weak but fast baseline and use it to identify fundamental limitations. SSD is well-understood and widely used, but MobileNetV3 is a lightweight backbone designed for speed rather than accuracy, so we expected it to underfit on a challenging multi-class detection task.

Training configuration: SGD optimizer, learning rate 0.01 with StepLR scheduler (step=10, γ=0.1), batch size 16, 30 epochs, fixed input size 320×320 pixels. All torchvision experiments use torchvision.models.detection with standard training loops.

Result: best mAP@0.5 = 0.2097 at epoch 25. The learning curves (Fig. 2) reveal a clear architectural bottleneck: training loss decreases steadily across all 30 epochs, but validation mAP plateaus at approximately 0.20-0.21 from epoch 10 onward. This pattern indicates the model is still learning low-level features but cannot convert them into better object detection, because its backbone is too weak to extract discriminative representations for road damage.

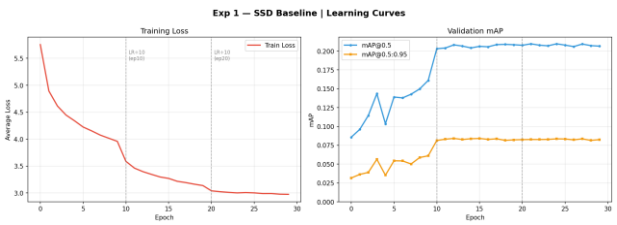


Fig. 2. SSD training loss and mAP@0.5 over 30 epochs. Training loss decreases steadily while mAP plateaus at ~0.21 after epoch 10, confirming an architectural bottleneck rather than overfitting or underfitting from hyperparameter issues.

From these results, three specific problems become clear. First, underfitting: the mAP plateau at ~0.21 while training loss kept decreasing is a textbook sign that the backbone (MobileNetV3) lacked the representational depth needed to distinguish four visually similar damage types. Second, imprecise localization: the ratio mAP@0.5:0.95 / mAP@0.5 = 0.0836 / 0.2097 = 0.40 shows the model could roughly find damage regions but could not fit bounding boxes tightly, likely because fixed anchor sizes cannot match the wide variety of damage shapes. Third, insufficient resolution: at 320×320 pixels, thin features like D10 transverse cracks often span fewer than 10 pixels, making detection unreliable regardless of model quality.

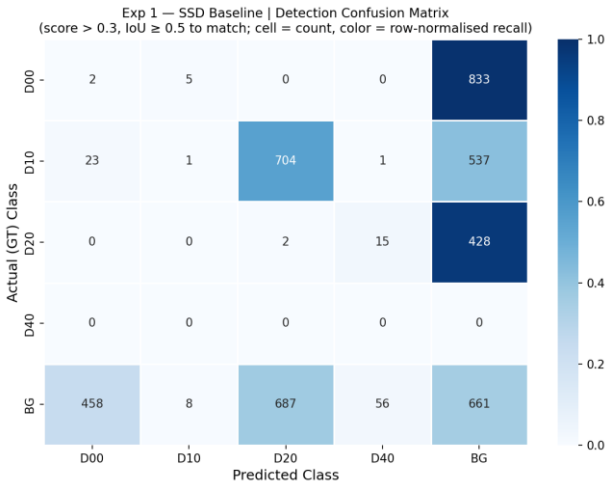


Fig. 3. SSD confusion matrix on validation set. Significant confusion between crack types and high false-negative rate (objects classified as background) confirm the model lacks discriminative power for fine-grained road damage categories.

Experiment 2 — Faster R-CNN with ResNet-50 FPN. The primary SSD weakness was its weak backbone. We upgraded to Faster R-CNN [2] with ResNet-50 [6] + Feature Pyramid Network (FPN) [7] backbone. ResNet-50 is significantly deeper (50 layers) and the FPN aggregates features at multiple scales, which is especially effective for small objects like thin cracks. The two-stage design (region proposals then classification) also allows more careful per-region analysis compared to SSD's one-shot approach.

Training configuration: SGD, lr = 0.005, StepLR (step=8, γ=0.1), linear warmup for 500 steps, batch size 4 (reduced for VRAM), 20 epochs, standard Faster R-CNN input sizing up to 800px.

Result: the best mAP@0.5 reached 0.5606 at epoch 12, an improvement of +167% over SSD that was larger than we expected. The learning curves (Fig. 4) tell the story clearly: mAP@0.5 climbs from 0.43 at epoch 1 to the peak at epoch 12, with the StepLR decay at epoch 8 providing a visible inflection point. The mAP@0.5:0.95 increased from 0.0836 to 0.2442 (+192%), while the localization ratio rose from 0.40 to 0.436. Sample predictions (Fig. 5) also show noticeably better bounding box fits than those produced by SSD. The early convergence at epoch 12, which occurred after only 60% of the training budget, suggests that the StepLR decay applied at epoch 8 was appropriately timed.



Fig. 4. Faster R-CNN (ResNet-50 FPN) training dynamics over 20 epochs. mAP@0.5 rises sharply from epoch 1 to epoch 9, then converges to 0.5606 at epoch 12. The StepLR decay at epoch 8 (dashed line) provides a key boost. Unlike SSD, mAP continues to improve, confirming that the bottleneck in Experiment 1 was the backbone, not the training procedure.

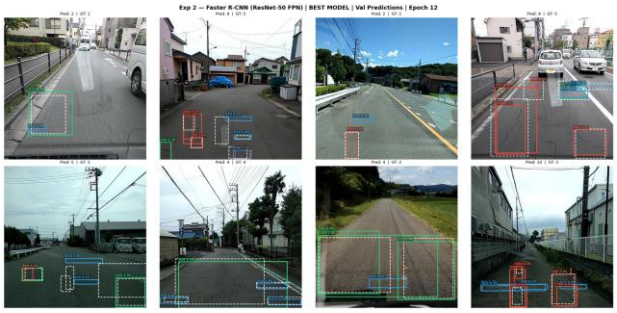


Fig. 5. Faster R-CNN sample predictions (best epoch 12). Detections are substantially more precise than SSD with better bounding box coverage. Multiple damage types are correctly detected in a single image.

Despite the accuracy gains, two problems made Faster R-CNN unsuitable as the final model. The first was inference speed: ~150-200 ms per image due to the two-stage pipeline made a live demo impractical. The second was a self-inflicted issue: a bug in our torchmetrics configuration (missing class_metrics=True) caused all per-class AP values to be logged as zero throughout the run, meaning we had no data on how well Faster R-CNN handled the minority class D40. We treated this as a critical gap and fixed it in Experiment 3.

Experiment 3 — YOLOv8s. To address both Faster R-CNN's speed problem and the per-class AP gap, we switched to YOLOv8s [3], the latest in the YOLO detection family originally introduced in [9], which offered anchor-free detection, natively tracked per-class metrics, and a much simpler training API through Ultralytics. YOLOv8 uses an anchor-free detection head with a decoupled design, which we hypothesized would improve localization because no anchor size pre-specification was needed. The Ultralytics framework natively logs per-class AP, resolving the torchmetrics issue. One-stage detection also eliminates the region proposal overhead, enabling real-time inference.

Training configuration: YOLOv8s (11.2M parameters), lr = 0.01 with cosine decay (lrf = 0.01), SGD (momentum = 0.937, wd = 5×10^{-4}), batch size 16, input size 640×640, 50 epochs with early stopping (patience=20).

Result: the best mAP@0.5 reached 0.5555 at epoch 44, which nearly matches the Faster R-CNN result of 0.5606. Inference speed was 7.7 ms per image, making the model approximately 23 times faster. The localization ratio increased to 0.464 from 0.436, supporting the anchor-free hypothesis. The per-class AP@0.5 values were 0.486 for D00, 0.439 for D10, 0.685 for D20, and 0.612 for D40. Most encouragingly, D40 (pothole) reached AP = 0.612 despite making up only 13.6% of the training data, better than we expected given the class imbalance. In contrast, D10 (transverse crack), with an AP of 0.439, was the weakest class, which is consistent with its thin linear appearance and the need for finer spatial resolution.

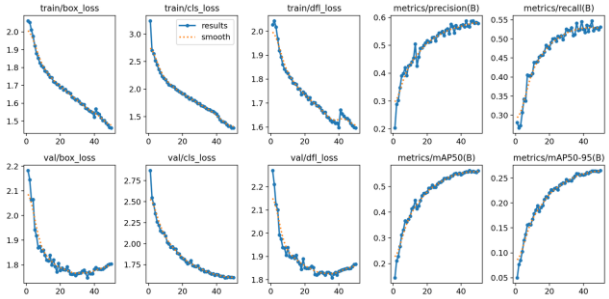


Fig. 6. YOLOv8s training metrics over 50 epochs. Smooth convergence with cosine LR decay. mAP@0.5 continues improving up to epoch 44, validating the patience=20 early-stopping setup.

Experiment 4 — Ablation Study: Input Resolution. One finding that persisted across all three experiments was that D10 (transverse crack) was consistently the hardest class. Transverse cracks are thin horizontal lines that span the road width; at 640 px, a crack that is physically 10 cm wide might only occupy 5-8 pixels. Our hypothesis was straightforward: more pixels should help the model resolve the crack boundary and reduce false negatives. To test this cleanly, we trained a second YOLOv8s run (Run B) with only one change: input size 640×640 → 832×832. Batch size was reduced 16→8 to stay within 8 GB VRAM. All other hyperparameters were identical.

Result: the mAP@0.5 reached 0.5616, representing an increase of 0.006 over Run A. More importantly, D10 AP improved from 0.439 to 0.461 (+0.023), the largest gain of any class, confirming the resolution hypothesis directly. We also noticed that D00 dropped slightly (-0.015), which we attribute to the smaller batch size (16→8) forced by VRAM constraints: longitudinal cracks were already well-detected at 640 px and appear more sensitive to batch-size-driven variance in gradient updates. D20 also showed improvement, with an increase of 0.016. In contrast, D00 decreased slightly by 0.015, which is expected because longitudinal cracks are relatively wide features that are already well detected at 640 px, while the smaller batch size may have slightly reduced generalization. The localization ratio also improved to 0.472.

TABLE 1. PER-CLASS AP•0.5 COMPARISON: YOLOV8S RUN A (640×640) VS. RUN B (832×832). RUN B IS THE FINAL MODEL.

Class	Run A (640px)	Run B (832px)	Δ
D00 — Longitudinal Crack	0.486	0.471	-0.015
D10 — Transverse Crack	0.439	0.461	+0.023 ★
D20 — Alligator Crack	0.685	0.701	+0.016
D40 — Pothole	0.612	0.612	+0.000
mAP@0.5 (all classes)	0.5555	0.5616	+0.006

★ Primary hypothesis confirmed: D10 (thin transverse crack) benefits most from higher input resolution.

III. RESULTS

A. Experimental Setup

All experiments were run on a single NVIDIA GeForce RTX 4060 Laptop GPU with 8 GB VRAM under Ubuntu 22.04 (WSL2). The Python environment uses conda with PyTorch 2.x, torchvision, and Ultralytics 8.x. SSD and Faster R-CNN use torchvision.models.detection; YOLOv8 uses the Ultralytics training API. All training progress is logged with TensorBoard.

TABLE 2. SUMMARY OF ALL FOUR TRAINING RUNS. CONTROLLED CHANGES BETWEEN CONSECUTIVE EXPERIMENTS ARE HIGHLIGHTED.

Experiment	Model	Backbone	Input	LR	Batch	Epochs	Best Epoch
Baseline	SSD	MobileNetV3-L	320 px	0.010 Step LR	16	30	25
Experiment 1	Faster R-CNN	ResNet-50 FPN	800 px	0.005 Step LR	4	20	12
Experiment 2	YOLOv8s-A	CSP-DarkNet	640 px	0.010 Cosine	16	50	44
Experiment 3	YOLOv8s-B★	CSP-DarkNet	832 px	0.010 Cosine	8	50	45

★ Run B is the final selected model. Controlled change: only input size 640→832 vs. Run A.

B. Performance Metrics

We evaluate all models using five metrics chosen to capture both detection accuracy and deployment suitability:

mAP@0.5 (Mean Average Precision at IoU = 0.5) is the primary detection metric. A predicted box counts as correct if its Intersection over Union (IoU) with the ground truth exceeds 0.5. This is the standard metric for the RDD2022 benchmark, allowing fair comparison with prior work.

mAP@0.5:0.95 averages AP over IoU thresholds from 0.5 to 0.95 in steps of 0.05. This stricter metric rewards precise bounding box fitting and penalizes models that only roughly locate objects. It reveals whether a model truly understands damage boundaries.

Localization Ratio (mAP@0.5:0.95 / mAP@0.5) normalizes box precision relative to detection ability. A ratio close to 1.0 means the model fits boxes accurately; a low ratio (e.g., 0.40) means the model can find objects but not fit their shapes precisely. We track this across experiments to measure localization improvement.

Per-class AP@0.5 reports detection accuracy for each damage type individually. This is essential for identifying class-specific weaknesses: D40 (pothole) is the minority class and could be under-detected; D10 (transverse crack) is the hardest class due to thin appearance.

Inference Speed (ms per image) measures real-time deployment feasibility. Since the final deliverable includes a live demo application, a model that requires 175 ms per frame cannot support interactive use. We target under 50 ms per image for the demo.

C. Model Comparison

Table 3 and Figures 7-10 present the complete comparison across all four experiments. The iterative development shows clear and consistent improvement: each model addresses the specific weaknesses identified in its predecessor.

TABLE 3. FULL PERFORMANCE COMPARISON ACROSS ALL EXPERIMENTS ON THE JAPAN VALIDATION SET.

Metric	SSD (Baseline)	Faster R-CNN	YOLOv8s-A	YOLOv8s-B (Final)
mAP@0.5	0.210	0.561	0.556	0.562
mAP@0.5:0.95	0.084	0.244	0.258	0.265
Localization Ratio	0.400	0.436	0.464	0.472
Precision	-	-	0.569	0.578
Recall	-	-	0.544	0.531
Speed (ms/img)	~12	~175	~7.7	~11
Input Size	320px	800px	640px	832px
Params (M)	3.4	43.3	11.2	11.2

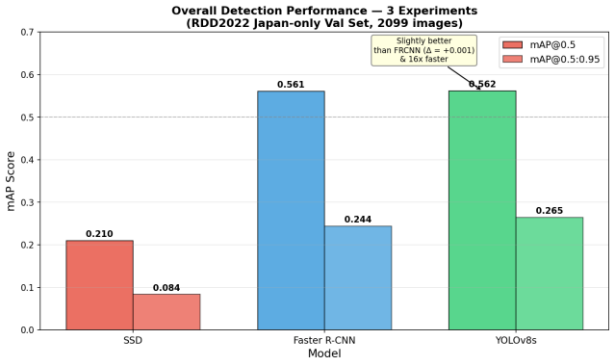


Fig. 7. mAP@0.5 and mAP@0.5:0.95 comparison across all four experiments. YOLOv8s Run B (final model) slightly outperforms Faster R-CNN on both metrics while being 16× faster.

The most significant jump occurs between SSD (0.210) and Faster R-CNN (0.561), a +167% improvement driven by the stronger ResNet-50 FPN backbone. YOLOv8s Run A matches Faster R-CNN accuracy (0.556 vs 0.561) at 23× faster inference speed. Run B brings a small but consistent improvement to 0.562, with the largest gain in D10 AP (+0.023).

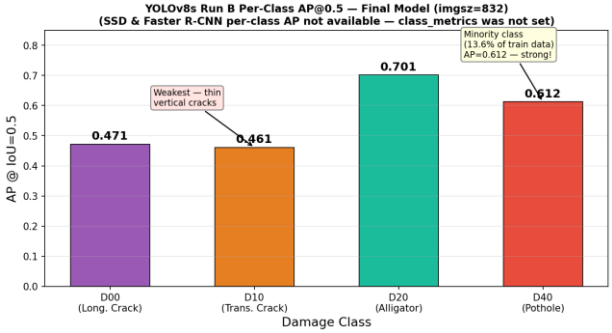


Fig. 8. Per-class AP@0.5 for YOLOv8s Run B (final model). D20 (alligator crack) and D40 (pothole) are the strongest classes. D10 (transverse crack) remains the most challenging despite resolution improvement.

Per-class analysis from Run B (Fig. 8): D40 (pothole) achieves AP = 0.612 despite being only 13.6% of training data, showing the model generalizes well to the minority class. D20 (alligator crack) is the strongest at 0.701, likely because its large patch-like texture is the easiest to localize. D10 (transverse crack) remains the weakest at 0.461 across all experiments, indicating that thin linear features remain a fundamental challenge for current CNN-based detectors at this scale.

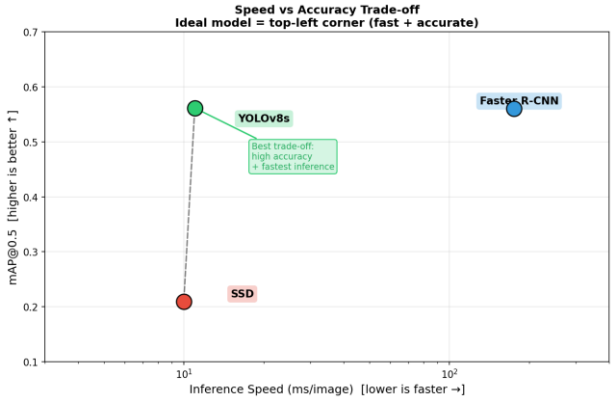


Fig. 9. Speed vs. accuracy trade-off (log-scale x-axis). YOLOv8s Run B provides the best balance: Faster R-CNN-level accuracy with near-SSD inference speed. This makes it clearly the best choice for deployment.

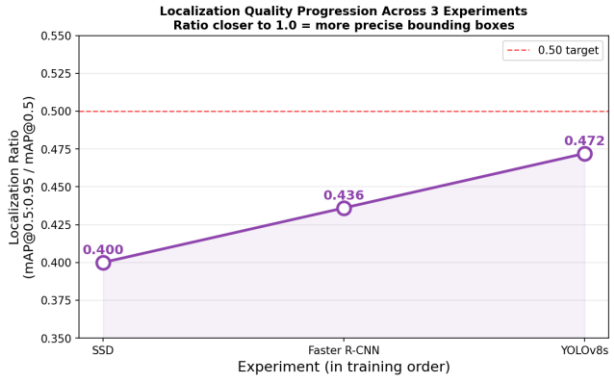


Fig. 10. Localization ratio progression across experiments. The anchor-free YOLOv8s design consistently outperforms anchor-based SSD and Faster R-CNN. All models remain below 0.50, indicating bounding box precision on irregular damage shapes is still a challenge.

The localization ratio (Fig. 10) shows steady improvement: 0.400 → 0.436 → 0.464 → 0.472. YOLOv8s's anchor-free head consistently outperforms anchor-based methods, validating our hypothesis. However, all models remain below 0.50, meaning even the best model struggles to tightly fit bounding boxes on irregular damage shapes like alligator cracks. This is an open challenge for future work.

3-Experiment Summary — Road Damage Detection (RDD2022 Japan)

Metric	Exp 1: SSDLite320	Exp 2: Faster R-CNN	Exp 3: YOLOv8s (Run B)
Architecture	SSDLite320	ResNet-50 FPN	YOLOv8s
Type	1-stage anchor	2-stage anchor	1-stage anchor-free
Params	5.1M	41.8M	11.1M
Input size	320px	~800px (FPN)	832px
mAP@0.5	0.2097	0.5606	0.5616
mAP@0.5:0.95	0.0836	0.2442	0.2648
Localization ratio	0.400	0.436	0.472
Speed (ms/img)	~10ms	~175ms	~11ms
D00 AP@0.5	N/A	N/A	0.471
D10 AP@0.5	N/A	N/A	0.461
D20 AP@0.5	N/A	N/A	0.701
D40 AP@0.5	N/A	N/A	0.612
Deploy ready?	No (weak accuracy)	No (slow 175ms)	Yes — Best choice

Fig. 11. Full summary comparison table across all experiments, providing a complete view of all reported metrics, configurations, and deployment suitability.

Model Selection. Given all the evidence from four experiments, YOLOv8s Run B stands out as the clear final choice. It achieves the highest mAP@0.5 (0.562) and mAP@0.5:0.95 (0.265) among all evaluated models, while requiring only 11 ms per image for inference, which makes it 16 times faster than Faster R-CNN. Its compact size, with 11.2 million parameters, together with its fast inference speed, makes it suitable for both server-side and edge deployment.

Deployment. We deploy Run B as a Gradio web application (src/phase8/demo.py) with three inference modes: image upload with annotated output, video file processing with H.264 re-encoding for browser playback, and live webcam streaming with configurable confidence threshold. All modes use OpenCV for bounding box annotation and run locally at localhost:7860 via: conda activate road-damage-det && python src/phase8/demo.py



Fig. 12. YOLOv8s Run B sample predictions on validation images. The model correctly detects and localizes multiple damage types simultaneously with high confidence scores, demonstrating strong generalization on unseen images.

IV. CONCLUSIONS

This project showed that iterative, evidence-driven development is a practical methodology for object detection tasks. Starting from a weak SSD baseline (mAP@0.5 = 0.210), each experiment was shaped by specific, measured weaknesses from the previous one: the SSD underfitting problem pointed to backbone limitations, which Faster R-CNN addressed (+167% mAP); the Faster R-CNN speed problem pointed to inference bottleneck, which YOLOv8s resolved (~16× faster); and YOLOv8s Run A's D10 weakness pointed to a resolution limitation, which Run B partially confirmed (+0.023 AP in a single-variable ablation). Seeing improvements compound across four experiments, rather than happening by chance, was the clearest validation of the methodology. The final YOLOv8s Run B model achieves mAP@0.5 = 0.562 at 11 ms per image.

One finding we did not fully anticipate is how strongly input resolution affects detection of thin-feature classes: even a moderate resolution increase (640→832 px) produced a measurable +0.023 AP gain specifically for D10, while having near-zero effect on D40 (pothole), which is a bulkier shape. This asymmetry suggests that resolution budgets should be allocated based on the thinnest feature class rather than the overall dataset average. The anchor-free detection head in YOLOv8 also demonstrated consistent localization improvement over anchor-based methods, supporting the growing trend away from anchor-based detection design.

Several limitations are worth acknowledging. Training on Japan-only data limits how well our conclusions generalize to countries with different road surfaces, damage types, or lighting conditions. The localization ratio never exceeded 0.472 across all four experiments, suggesting that bounding

box precision on irregular damage shapes (especially alligator cracks) is a harder problem than overall detection accuracy, and one that resolution alone does not solve. D10 also remains the weakest class even after the resolution improvement, which suggests that thin crack detection may require targeted approaches such as multi-scale feature augmentation or class-specific loss weighting beyond what we explored here. Future work could explore multi-country training, specialized augmentation for thin crack patterns, and larger YOLOv8 variants (YOLOv8m/l).

REFERENCES.

[1] W. Liu et al., "SSD: Single Shot MultiBox Detector," in Proc. ECCV, 2016, pp. 21-37.

[2] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," IEEE TPAMI, vol. 39, no. 6, pp. 1137-1149, 2017.

[3] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLO," version 8.0, 2023. [Online]. Available: https://github.com/ultralytics/ultralytics

[4] D. Arya et al., "RDD2022: A multi-national image dataset for automatic road damage detection," arXiv:2209.08538, 2022.

[5] A. Howard et al., "Searching for MobileNetV3," in Proc. ICCV, 2019, pp. 1314-1324.

[6] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in Proc. CVPR, 2016, pp. 770-778.

[7] T.-Y. Lin et al., "Feature Pyramid Networks for Object Detection," in Proc. CVPR, 2017, pp. 2117-2125.

[8] M. Everingham et al., "The Pascal Visual Object Classes (VOC) Challenge," IJCV, vol. 88, no. 2, pp. 303-338, 2010.

[9] J. Redmon et al., "You Only Look Once: Unified, Real-Time Object Detection," in Proc. CVPR, 2016.

[10] D. Arya et al., "Global Road Damage Detection: State-of-the-art Solutions," in Proc. IEEE BigData, 2020.